

Fase 1: Fundamentação Teórica e Diretrizes de Estudo

Antes da implementação em código, o desenvolvimento do modelo de inteligência artificial requer a compreensão de técnicas específicas para Redes Neurais Ternárias (TNN). O treinamento desse tipo de modelo diverge fundamentalmente das redes neurais tradicionais.

- **Adoção de *Quantization-Aware Training* (QAT):** Em contrapartida ao método convencional de treinar em Ponto Flutuante (32-bits) e converter posteriormente para resoluções menores (*Post-Training Quantization* - PTQ), a abordagem ternária exige o uso de QAT. O PTQ causa degradação severa na precisão de redes com poucos bits. O QAT, por sua vez, simula a perda de precisão durante o próprio treinamento, forçando a rede a ajustar seus pesos considerando as restrições dos valores -1, 0 e 1.
- **Implementação do *Straight-Through Estimator* (STE):** A função de quantização que converte valores para -1, 0 e 1 possui a característica de uma função degrau, não sendo contínua. Consequentemente, o cálculo tradicional do gradiente (*Backpropagation*) falha, pois a derivada é nula na maior parte da função. O conceito de STE deve ser aplicado como um recurso matemático para estimar gradientes válidos durante o treinamento, permitindo a atualização dos pesos.
- **Definição da Arquitetura (MLP):** A topologia inicial da rede deve ser baseada em *Multi-Layer Perceptron* (MLP), empregando exclusivamente camadas densas (*Fully Connected*). A utilização de Redes Convolucionais (CNNs) deve ser evitada nesta etapa do projeto para minimizar a complexidade do roteamento no projeto de hardware em Verilog, mantendo o foco nas operações de multiplicação matricial pura.

Fase 2: Implementação e Treinamento do Modelo

O treinamento da rede não exige o desenvolvimento de algoritmos de quantização do zero. Recomenda-se a utilização de frameworks já estabelecidos na literatura acadêmica e na indústria. O desenvolvimento pode seguir por uma de duas vias principais:

- **Opção A: Framework Larq (Ecossistema Keras / TensorFlow)**
 - **Descrição:** Biblioteca *Open Source* construída sobre o TensorFlow, dedicada a Redes Binárias (BNNs) e Ternárias (TNNs). Destaca-se pela facilidade de implementação.
 - **Implementação:** Substituição das chamadas tradicionais `tf.keras.layers.Dense` pela sua contraparte quantizada `larq.layers.QuantDense`, procedendo com a parametrização do quantizador ternário para os pesos da respectiva camada.
- **Opção B: Framework Brevitas (Ecossistema PyTorch)**
 - **Descrição:** Ferramenta avançada para pesquisa, com foco em integração de modelos quantizados em FPGAs. Costuma ser o padrão em publicações da área.
 - **Implementação:** Utilização da classe `qnn.QuantLinear`, configurando a precisão dos pesos (`weight_bit_width=2`) e o algoritmo de quantização para o comportamento ternário.

- **Especificações do Modelo e Entregável:** O treinamento será realizado utilizando o dataset MNIST, padronizando um vetor de entrada de tamanho 784 (referente às dimensões 28x28 achatadas). O artefato resultante desta fase deve ser um script Python capaz de treinar o modelo até atingir uma acurácia mínima de 95%, garantindo que os pesos da rede operem exclusivamente no conjunto de valores {-1, 0, 1}.

Fase 3: Extração de Pesos e Empacotamento (Interface Python/C)

Para a execução de inferência diretamente no sistema embarcado (*Bare-Metal Inference*), sem a sobrecarga de bibliotecas como o TFLite, os dados treinados precisam ser processados e empacotados. O seguinte pipeline deve ser implementado via script Python:

1. **Extração dos Tensores:** As matrizes de pesos resultantes do treinamento devem ser extraídas das camadas da rede. Estes valores estatísticos devem ser rigorosamente mapeados e forçados para os números inteiros correspondentes (-1, 0 e 1).
2. **Codificação Binária:** A representação dos três valores deve ser definida em um formato de 2 bits. Exemplo de codificação sugerida:
 - 00 representa o valor numérico 0.
 - 01 representa o valor numérico 1.
 - 11 representa o valor numérico -1 (representação baseada em complemento de 2).
3. **Algoritmo de Empacotamento (Packing):** Tendo em vista a otimização de banda de memória em um barramento de 32 bits (ou 64 bits), o envio unitário de pesos de 2 bits é ineficiente. O algoritmo deve agregar 16 pesos de 2 bits por vez, empregando operações de deslocamento de bits (*shift bitwise <<*) para compor um único registrador do tipo inteiro sem sinal de 32 bits (`uint32_t`).
4. **Exportação do Arquivo Header (weights.h):** O script deve serializar o conjunto empacotado em um arquivo textual formatado no padrão da linguagem C, resultando em estruturas de dados prontas para compilação.

C

```
// weights.h gerado automaticamente  
const uint32_t layer1_weights[] = { 0x4F0A11B2, 0x99C2001F, ... };
```

Fase 4: Desenvolvimento da Aplicação de Inferência (User Space)

A etapa final contempla a codificação do programa executável na linguagem C, projetado para operar sobre a distribuição Linux executada pelo processador RISC-V.

- **Ambiente de Compilação (Cross-Compiler):** O código fonte não pode ser compilado nativamente no host (arquitetura x86). O binário deve ser gerado utilizando uma *Cross-Compiler Toolchain* configurada para o ambiente alvo (ex: `riscv64-unknown-linux-gnu-gcc`), garantindo a execução do binário na arquitetura RISC-V.
- **Fluxograma e Rotinas do Código:**

- **Inclusões:** Importação do arquivo `weights.h` (obtido na Fase 3) e da representação da imagem a ser inferida sob formato de array bidimensional/unidimensional.
- **Interface com o Driver de Dispositivo:** Acesso ao módulo gerenciador do hardware via bibliotecas POSIX do Linux.

C

```
int fd = open("/dev/npu_ternaria", O_RDWR);
```

- **Transmissão de Dados:** Escrita estruturada dos buffers contendo as amostras e os arrays de pesos empacotados para a NPU, empregando chamadas de sistema como `write()` ou `ioctl()`.
- **Sincronização:** Implementação de rotinas de bloqueio, interrupção ou verificação contínua (*polling*) a fim de identificar a finalização das operações da NPU na matriz.
- **Aquisição do Processamento:** Uso da chamada `read()` para resgatar o vetor resultante em memória correspondente ao final da camada da rede.
- **Benchmarking e Coleta de Métricas:** Etapa crítica para fundamentação dos resultados acadêmicos. Devem-se implementar funções das bibliotecas `<time.h>` ou `<sys/time.h>` (especificamente `gettimeofday()`) para cronometrar a operação com precisão de milissegundos ou microssegundos. O software deverá processar a mesma inferência matematicamente em *software puro* (via CPU) para estabelecer uma correlação comparativa de ganho de desempenho frente à NPU.